

Método de Runge-Kutta de 4° Orden

Resolución Numérica de Ecuaciones Diferenciales Ordinarias

Materia: Análisis Numérico — 2026
Integrantes: Tomas Villar | Joaquin Vanrrel

1. Introducción

Las ecuaciones diferenciales ordinarias (EDO) de primer orden de la forma $y' = f(x, y)$, $y(x_n) = y_n$ aparecen con frecuencia en problemas de Ingeniería en Informática: modelado térmico de procesadores, simulación de sistemas de control, análisis de rendimiento de redes, entre otros. En la mayoría de los casos prácticos no existe solución analítica cerrada, por lo que se recurre a métodos numéricos para aproximar la solución.

El Método de Euler y el de Taylor permiten resolver estos problemas, pero presentan errores de truncamiento locales del orden de h (Euler) o requieren evaluar derivadas de alto orden (Taylor). El **Método de Runge-Kutta de 4° Orden (RK4)** supera ambas limitaciones: alcanza un error de truncamiento local del orden $O(h^4)$, siendo notablemente más preciso sin necesidad de calcular derivadas de orden superior.

2. Fundamento Matemático

Dado un Problema de Valor Inicial (PVI):

$$dy/dx = f(x, y), \quad y(x_n) = y_n$$

El método RK4 construye la solución paso a paso. Conocido el punto (x_n, y_n) , el siguiente se calcula mediante la fórmula de recurrencia:

$$y_{n+1} = y_n + h \cdot \phi(x_n, y_n)$$

donde ϕ es un promedio ponderado de cuatro pendientes:

$$\phi = (k_1 + 2 \cdot k_2 + 2 \cdot k_3 + k_4) / 6$$

Cada pendiente estima la derivada en un punto distinto del intervalo:

$k_1 = f(x_n, y_n)$	pendiente al inicio del paso
$k_2 = f(x_n + h/2, y_n + (h/2) \cdot k_1)$	pendiente en el punto medio (usando k_1)
$k_3 = f(x_n + h/2, y_n + (h/2) \cdot k_2)$	pendiente en el punto medio (usando k_2)
$k_4 = f(x_n + h, y_n + h \cdot k_3)$	pendiente al final del paso

Orden:	4° — error local $O(h^4)$, error global $O(h^4)$
Autoarrancante:	no requiere valores previos, sólo la condición inicial
vs. Taylor:	no requiere calcular derivadas de orden superior
vs. Euler:	error mucho menor para el mismo tamaño de paso h

3. Aplicación a la Ingeniería en Informática

3.1 Gestión Térmica en Sistemas Computacionales

En Ingeniería en Informática, el comportamiento térmico de los componentes es un factor crítico en el diseño y operación de sistemas. Los procesadores, GPUs y otros chips disipan calor durante su funcionamiento, y su temperatura evoluciona de forma continua en el tiempo en respuesta a cambios en la carga de trabajo, la velocidad del ventilador o las condiciones del entorno.

Conocer con precisión cómo varía la temperatura permite:

- **Diseñar algoritmos de throttling:** reducir la frecuencia del procesador antes de alcanzar la temperatura crítica de apagado.
- **Optimizar sistemas de refrigeración:** dimensionar disipadores y ventiladores según la velocidad de enfriamiento requerida.
- **Simular escenarios de carga:** predecir el comportamiento térmico de un sistema embebido o servidor antes de fabricarlo o desplegarlo.

Todos estos fenómenos involucran una variable (temperatura) que cambia continuamente en función del tiempo, convirtiéndolos en PVI's resolubles con RK4.

3.2 Modelo Matemático: Ley de Enfriamiento de Newton

Para modelar el enfriamiento de un procesador se utiliza la **Ley de Enfriamiento de Newton**, que establece que la tasa de cambio de temperatura de un cuerpo es proporcional a la diferencia entre su temperatura y la del entorno:

$$dT/dt = -k \cdot (T - T_{\text{amb}})$$

T	temperatura del procesador en el instante t (°C)
T_amb	temperatura ambiente (°C)
k	constante de disipación térmica (min ⁻¹)
t	tiempo (minutos)

Esta ecuación es una EDO de la forma $y' = f(x, y)$, resoluble directamente con RK4. En casos más complejos —donde k varía con la temperatura o intervienen varias fuentes de calor— la EDO deja de tener solución analítica simple y el método numérico se vuelve la única herramienta práctica.

3.3 Ejemplo: Enfriamiento de un Procesador bajo Reducción de Carga

Se considera un procesador que alcanza **95 °C** bajo carga máxima sostenida. Al reducir la carga comienza a enfriarse. Se desea determinar cuánto tarda en bajar a **40 °C** (umbral habitual para desactivar el throttling), con:

- Temperatura inicial: $T(0) = 95$ °C
- Temperatura ambiente: $T_{\text{amb}} = 25$ °C
- Constante de disipación: $k = 0.05$ min⁻¹

La EDO del problema queda:

$$dT/dt = -0.05 \cdot (T - 25), \quad T(0) = 95$$

Esta EDO tiene solución analítica, lo que permite verificar la precisión del método:

$$T(t) = 25 + 70 \cdot e^{(-0.05 \cdot t)}$$

$$\text{Tiempo exacto para } T = 40 \text{ °C: } t = \ln(70/15) / 0.05 \approx 30.81 \text{ minutos}$$

Función ingresada en el programa:

$$f(x, y) = -0.05 \cdot (y - 25)$$

$x_0 = 0$	$y_0 = 95$	$x_{\text{final}} = 35$	$h = 1$
-----------	------------	-------------------------	---------

4. Resultados

Tabla comparativa RK4 vs. solución analítica — $f(t, T) = -0.05 \cdot (T - 25)$, $T(0) = 95$ °C, $h = 1$ min

Paso	t (min)	T_RK4 (°C)	T_exacta (°C)	Error absoluto
0	0.0	95.000000	95.000000	0.00e+00
1	1.0	91.586060	91.586060	1.81e-07
2	2.0	88.338620	88.338619	3.44e-07
3	3.0	85.249559	85.249558	4.91e-07
4	4.0	82.311153	82.311153	6.22e-07
5	5.0	79.516056	79.516055	7.40e-07
6	6.0	76.857276	76.857275	8.45e-07
7	7.0	74.328167	74.328166	9.37e-07
8	8.0	71.922404	71.922403	1.02e-06
9	9.0	69.633972	69.633971	1.09e-06
10	10.0	67.457147	67.457146	1.15e-06
11	11.0	65.386488	65.386487	1.21e-06
12	12.0	63.416816	63.416815	1.25e-06
13	13.0	61.543206	61.543204	1.29e-06
14	14.0	59.760973	59.760971	1.32e-06
15	15.0	58.065660	58.065659	1.35e-06
16	16.0	56.453029	56.453027	1.37e-06
17	17.0	54.919047	54.919045	1.38e-06
18	18.0	53.459878	53.459876	1.39e-06
19	19.0	52.071873	52.071872	1.40e-06
20	20.0	50.751562	50.751561	1.40e-06
21	21.0	49.495644	49.495642	1.40e-06
22	22.0	48.300977	48.300976	1.39e-06
23	23.0	47.164575	47.164574	1.38e-06
24	24.0	46.083596	46.083595	1.37e-06
25	25.0	45.055337	45.055336	1.36e-06
26	26.0	44.077227	44.077226	1.35e-06
27	27.0	43.146820	43.146818	1.33e-06
28	28.0	42.261789	42.261787	1.31e-06
29	29.0	41.419921	41.419920	1.29e-06
30	30.0	40.619112	40.619111	1.27e-06
31	31.0	39.857359	39.857358	1.25e-06
32	32.0	39.132757	39.132756	1.23e-06

Paso	t (min)	T_RK4 (°C)	T_exacta (°C)	Error absoluto
33	33.0	38.443495	38.443494	1.20e-06
34	34.0	37.787848	37.787847	1.18e-06
35	35.0	37.164177	37.164176	1.16e-06

* Fila resaltada: en t = 30 min la temperatura baja de 40.62 °C a 39.86 °C en t = 31 min, cruzando el umbral objetivo. Interpolando: t ≈ 30.81 minutos.

5. Descripción del Programa

El programa `Runge-Kutta.py` implementa RK4 de forma genérica, permitiendo resolver cualquier EDO de primer orden $y' = f(x, y)$.

obtener_funcion()	Solicita la expresión de $f(x, y)$ como texto y construye una función Python evaluable con <code>eval()</code> . Soporta sin, cos, tan, exp, log, sqrt.
obtener_condiciones()	Solicita x_0 , y_0 , x_{final} y h . Retorna los cuatro parámetros necesarios para definir completamente el PVI.
runge_kutta_4(f, x0, y0, h, n_pasos)	Núcleo del programa. Calcula k_1 , k_2 , k_3 , k_4 en cada iteración y acumula los puntos de la solución. Detecta divergencia y desbordamiento numérico.
mostrar_resultados(xs, ys, formula)	Imprime la tabla de resultados en formato tabulado con número de paso, x e y .
main()	Coordina el flujo completo con validación de entradas: fórmula evaluable, $x_{\text{final}} > x_0$ y $h > 0$.

6. Instrucciones de Ejecución

Requisito: Python 3.x instalado. No requiere librerías externas (sólo el módulo estándar `math`).

Linux / macOS	<code>python3 Runge-Kutta.py</code>
Windows	<code>python Runge-Kutta.py</code> (o bien: <code>py Runge-Kutta.py</code>)

Otros ejemplos de funciones válidas:

$y' = 2x + 1$	<code>2*x + 1</code>
$y' = \sin(x) \cdot y$	<code>sin(x) * y</code>
$y' = \exp(-x) - y$	<code>exp(-x) - y</code>
$y' = \frac{1}{2}(1+x)y^2$	<code>0.5 * (1 + x) * y**2</code>